

Tư duy Hướng đối tượng C# trong Phát triển Web Doanh nghiệp

Sự Chuyển dịch Tư duy



Từ Lập trình Thủ tục: Viết các hàm xử lý rời rạc (Nghệp dư).



Sang Thiết kế Đối tượng: Thiết kế các thực thể nghiệp vụ và luồng tương tác sống động (Kỹ sư Doanh nghiệp).

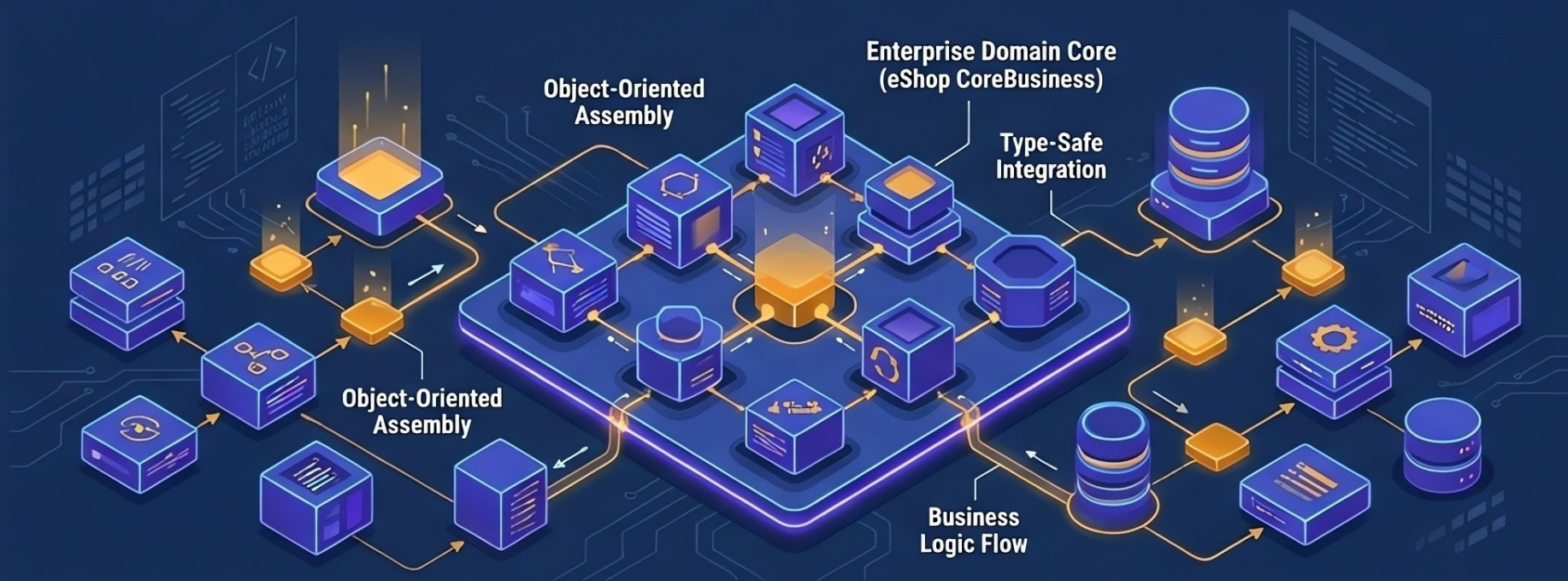
Mục tiêu & Nền tảng



Mục tiêu: Xây dựng tầng thực thể cốt lõi (CoreBusiness) vững chắc cho hệ thống eShop.



Nền tảng: Khai thác tối đa C# hiện đại với tính an toàn kiểu dữ liệu (Strict Type Safety) tuyệt đối.





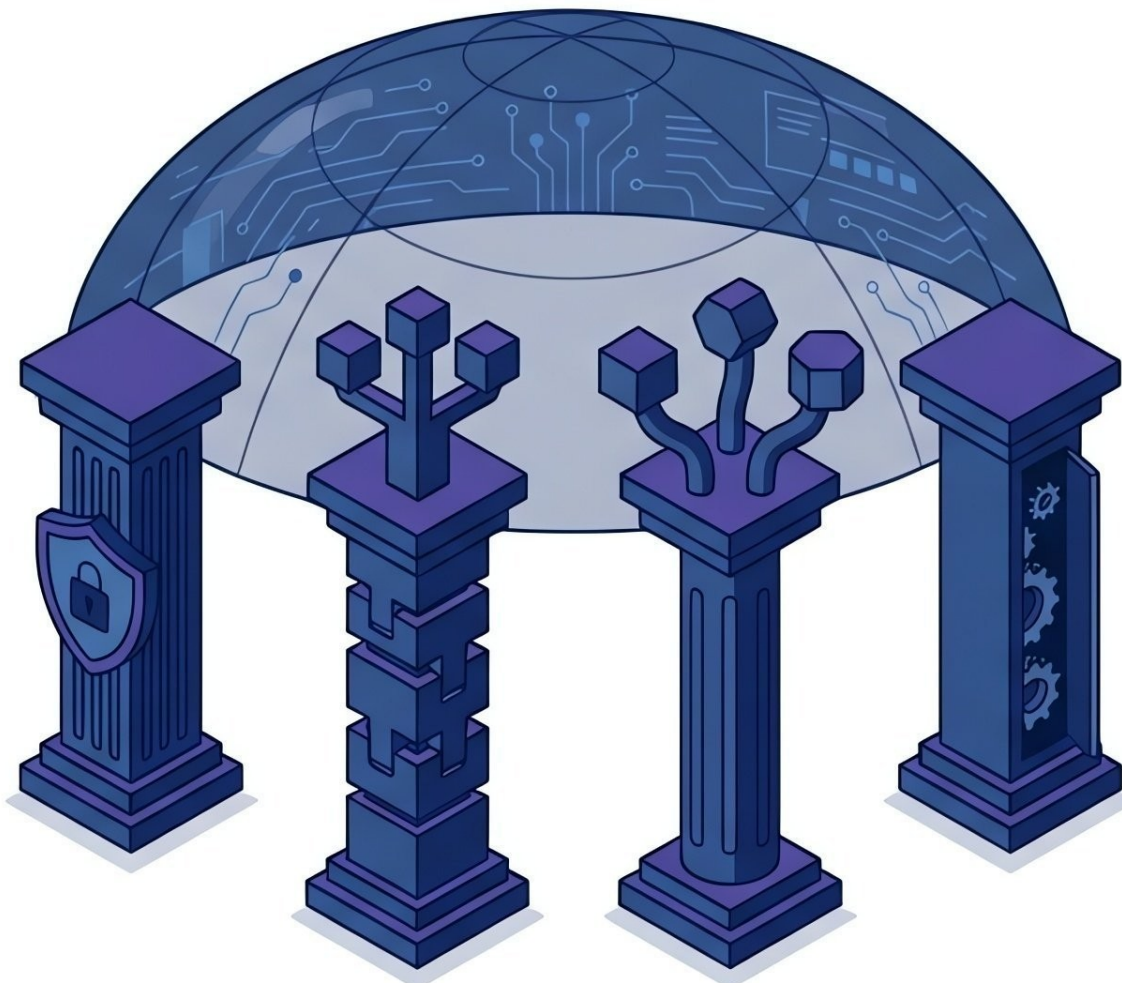
Đóng gói (Encapsulation)

Bảo vệ trạng thái thực thể.
Chỉ cho phép sửa đổi dữ liệu
qua các quy tắc nghiệp vụ hợp lệ

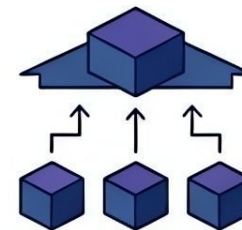


Đa hình (Polymorphism)

Định tuyến một lệnh thực thi
tới nhiều hành vi khác nhau tùy
theo từng ngữ cảnh đối tượng.

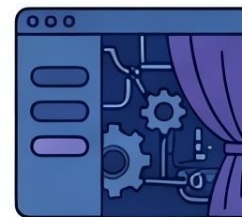


4 Trụ cột OOP qua Lăng kính Ứng dụng Web Thực tế



Kế thừa (Inheritance)

Tái sử dụng thuộc tính chung
(như BaseEntity) để tối ưu hóa
mã nguồn toàn hệ thống.



Trừu tượng (Abstraction)

Ẩn giấu chi tiết cài đặt phức
tạp đằng sau các giao diện
(Interfaces) tinh gọn.

Đóng gói & Quản lý Thuộc tính với C# Auto-Properties



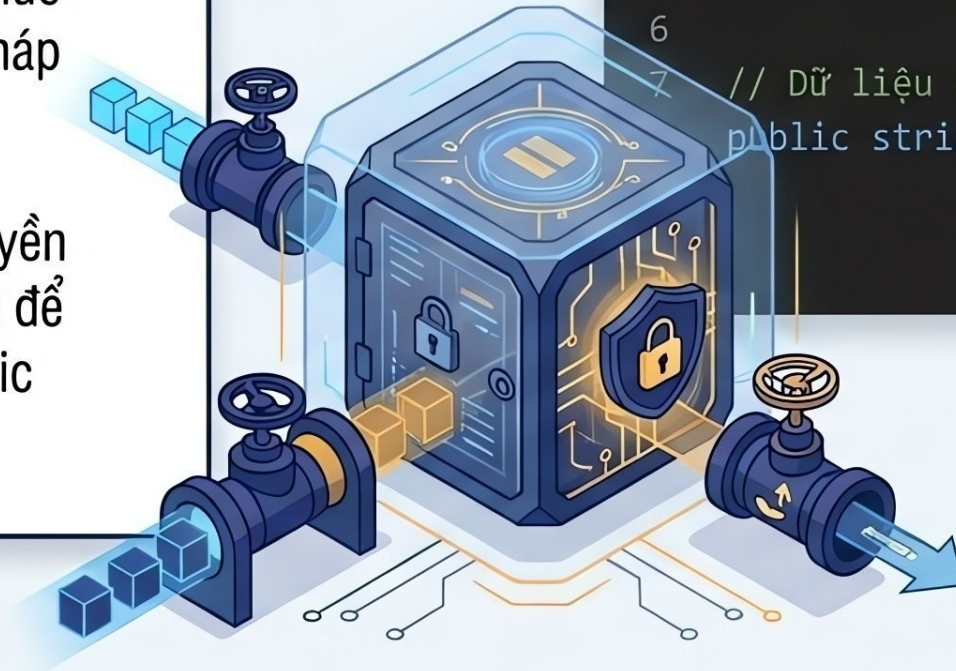
Cơ chế Auto-Property



Tích hợp biến riêng (private field) và cặp phương thức get / set vào một cú pháp tinh gọn duy nhất.

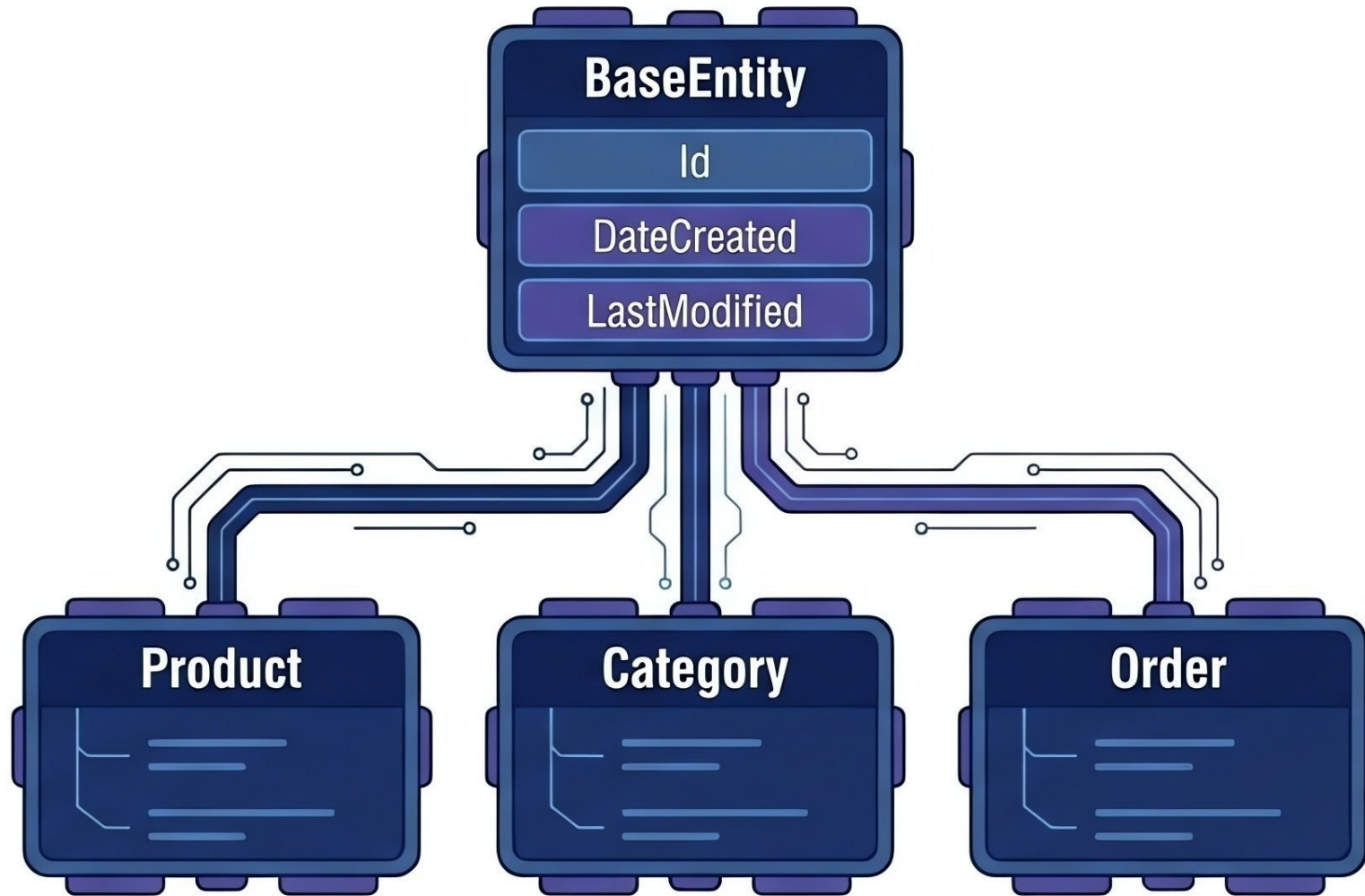


Kiểm soát chặt chẽ quyền ghi (Access Modifiers) để ngăn chặn 80% lỗi logic ngoài ý muốn.



```
1 // Tiêu chuẩn
2 public string Name { get; set; } = string.Empty;
3
4 // Đọc công khai, Ghi nội bộ
5 public int Id { get; private set; }
6
7 // Dữ liệu bất biến sau khởi tạo
8 public string SKU { get; init; }
```

Kế thừa & Chia sẻ Thuộc tính Dùng chung



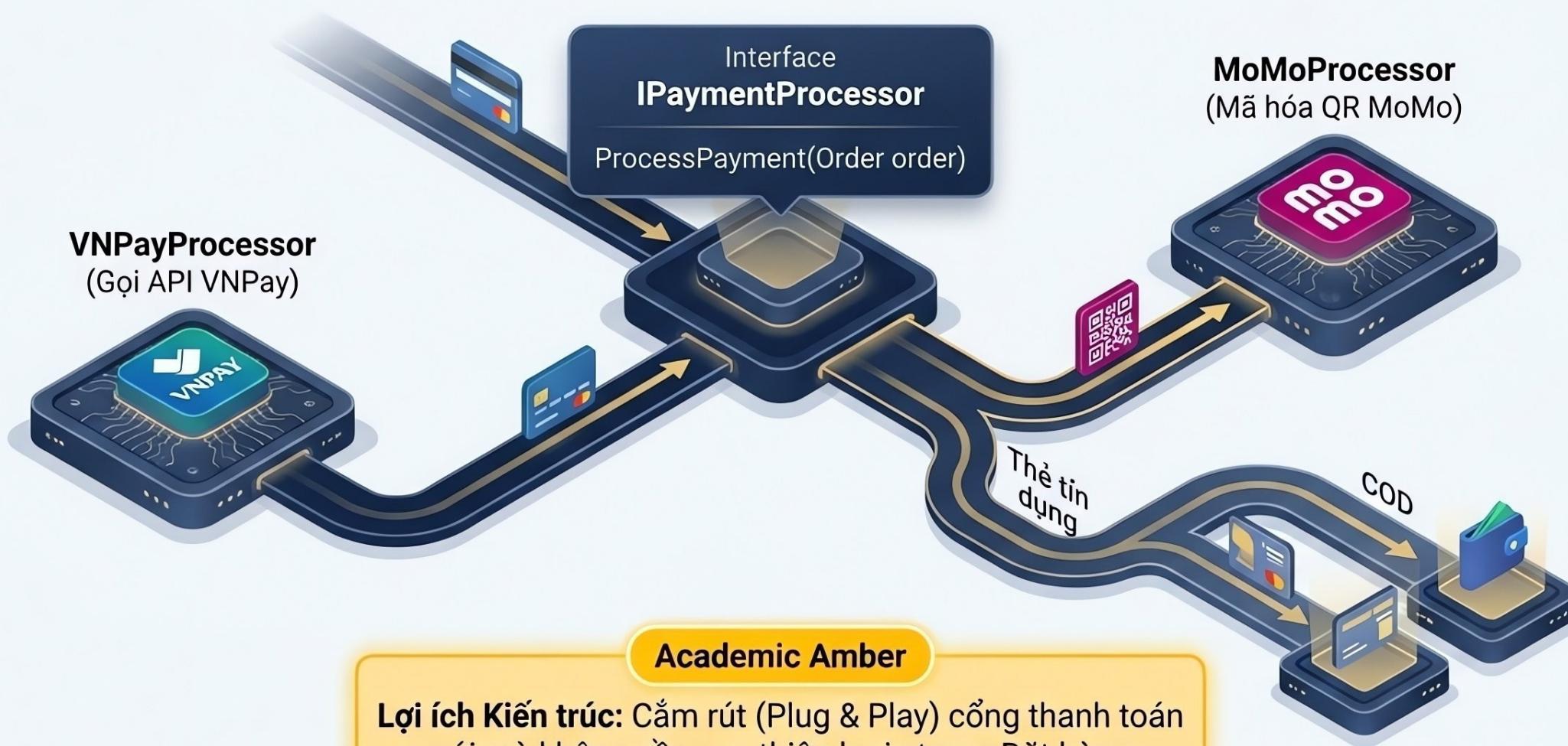
Kế thừa trực tiếp để tái sử dụng mã.

Nguyên tắc Kế thừa Hợp lý

-  **Bắt buộc:** Chỉ áp dụng khi có quan hệ bản chất IS-A (Ví dụ: Sản phẩm LÀ MỘT Thực thể).
-  **Cảnh báo:** Tránh lạm dụng kế thừa quá nhiều tầng gây phức tạp hóa kiến trúc.

Đa hình: Ứng dụng trong Đa cổng Thanh toán

Bài toán: eShop cần hỗ trợ linh hoạt VNPay, MoMo, Thẻ tín dụng và COD.



Lợi ích Kiến trúc: Cắm rút (Plug & Play) cổng thanh toán mới mà không cần can thiệp logic trang Đặt hàng.

Trừu tượng (Abstraction): Tách rời Ý niệm khỏi Triển khai

Tầng UI

Chỉ cần gọi lệnh: **IProductRepository.GetProducts()**
để nhận danh sách.

Tập trung vào câu hỏi:
Đối tượng này **LÀM ĐƯỢC GÌ?**
thay vì **Nó LÀM NHƯ THẾ NÀO?**

Tầng Kỹ thuật Lỗi

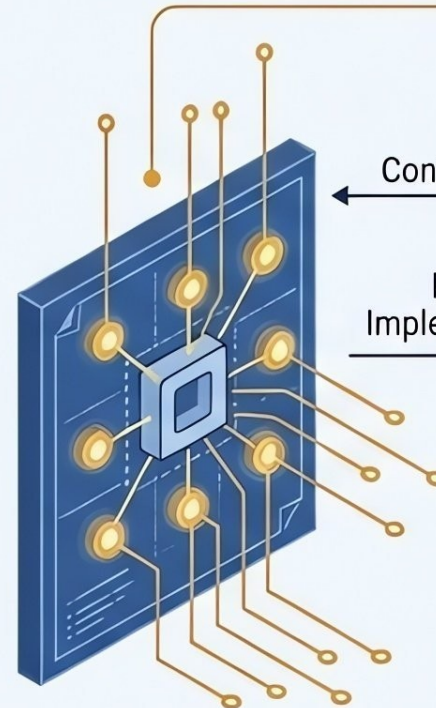
Hệ thống tự động quyết định lấy dữ liệu từ RAM, file JSON hay SQL Server.
UI hoàn toàn không bận tâm.



Phân biệt Cốt tử: Interface vs. Abstract Class

Interface (Bản Hợp đồng 100% Trừu tượng)

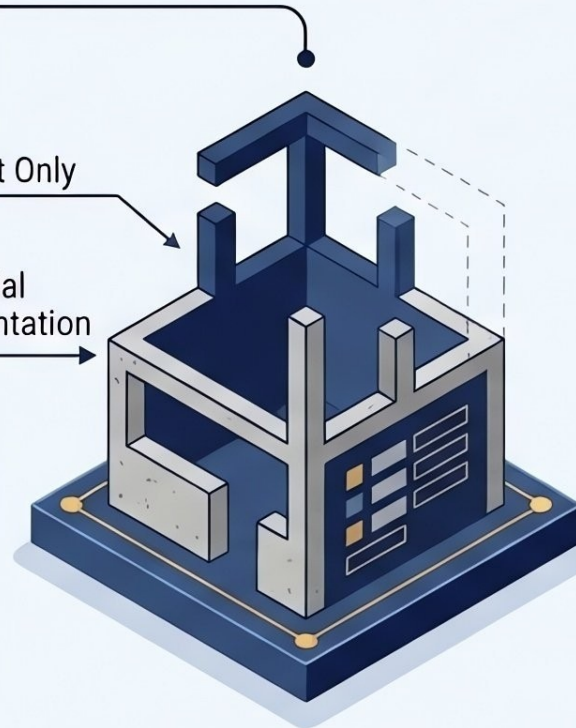
- Chỉ định nghĩa **danh sách phương thức**, không chứa **logic** thực thi hay trạng thái biến.
- Cho phép **Đa kế thừa** hành vi (Triển khai nhiều Interface cùng lúc).
- Ứng dụng: **IProductRepository, IShoppingCart**



INTERFACE SOCKET
Multiple Implementations

Abstract Class (Lớp Cơ sở Bán phần)

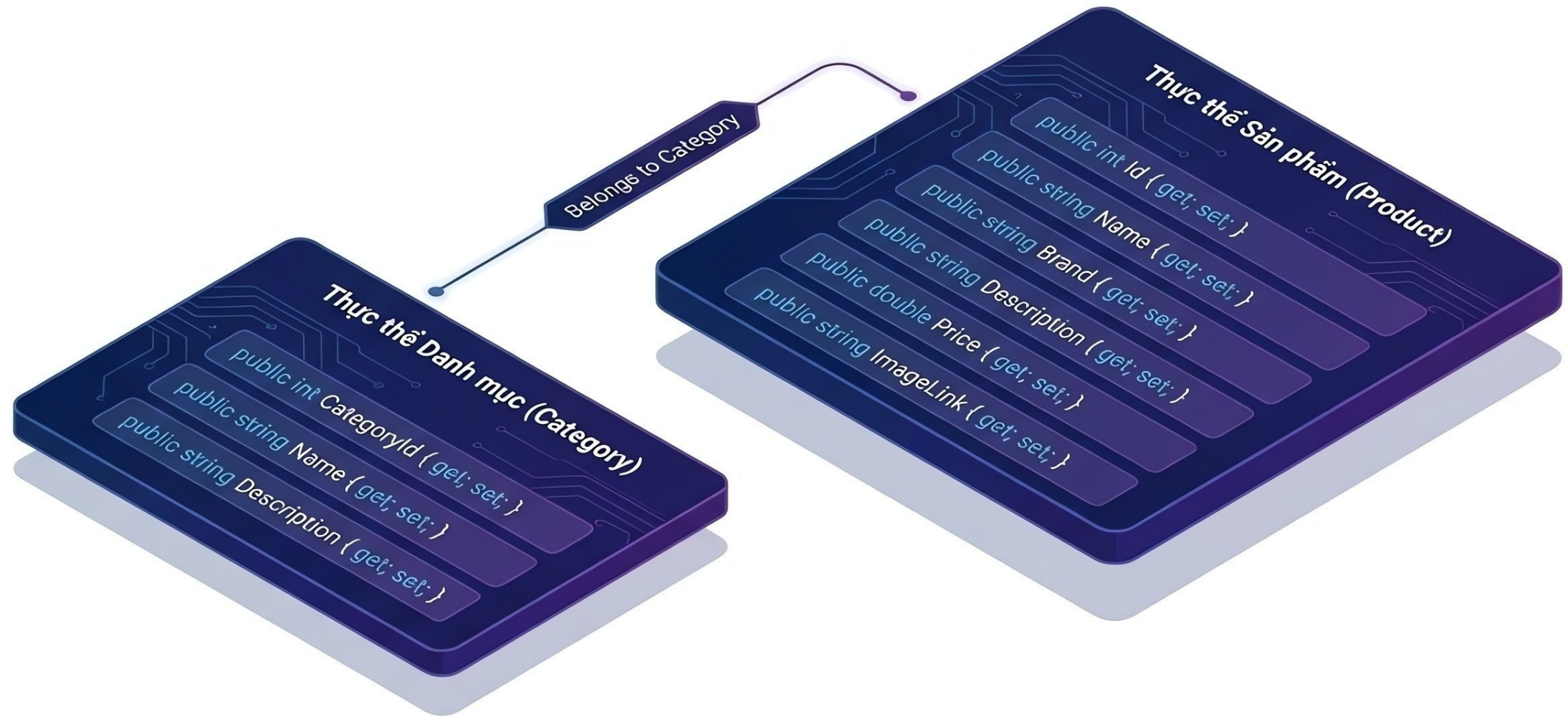
- Có thể chứa **mã nguồn thực thi chung** và các trường dữ liệu cụ thể.
- Giới hạn **Đơn kế thừa** (Chỉ được kế thừa từ một lớp duy nhất).
- Ứng dụng: **BaseEntity**



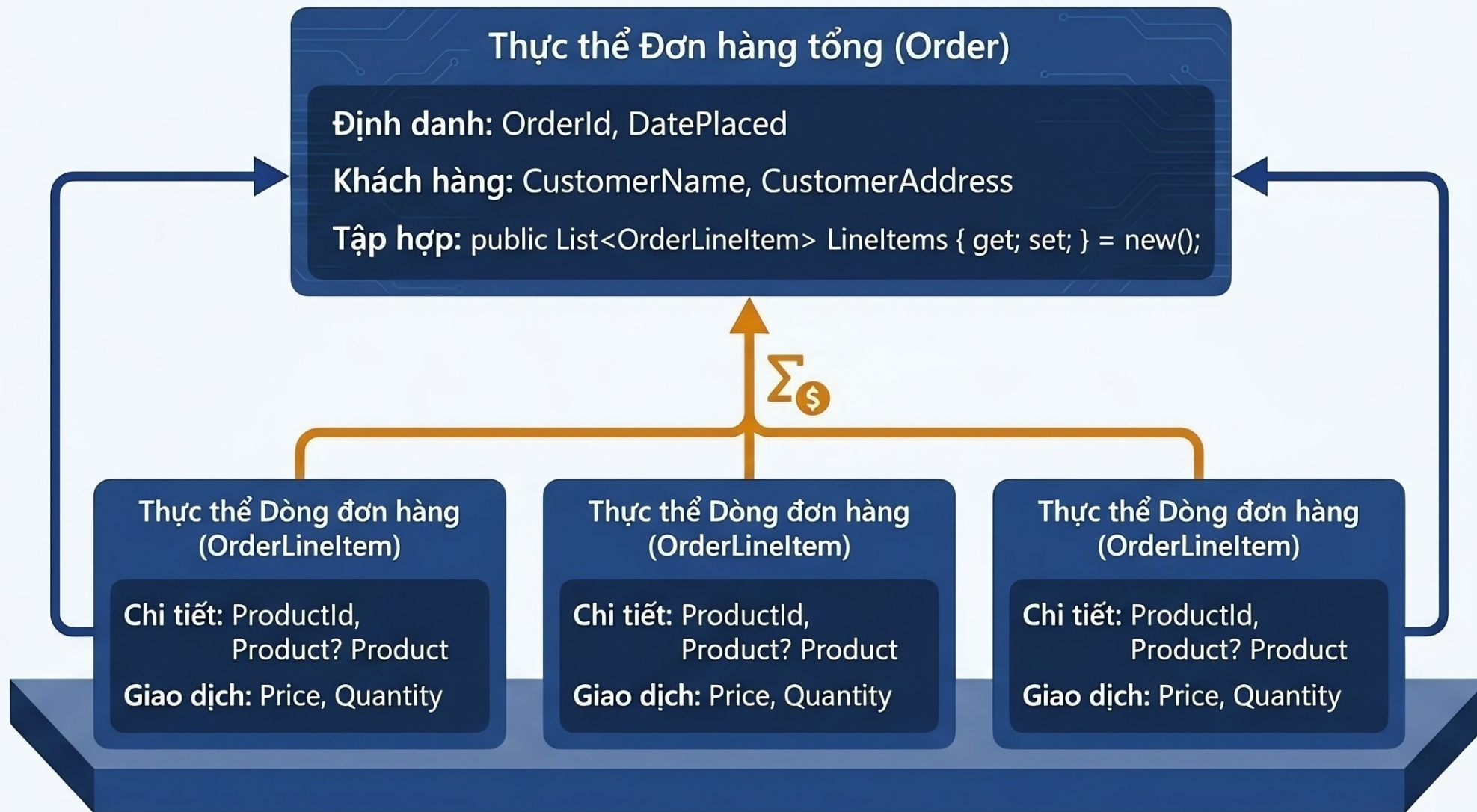
ABSTRACT FRAMEWORK
Partial Implementation & State

Contract Only
Partial Implementation

Thiết kế Thực thể Mẫu: Lớp Product và Category



Thiết kế Thực thể Giao dịch: Quan hệ 1-Nhiều



Quản lý Danh sách Đối tượng với C# Generics

Sức mạnh của <T>



- Bảo chứng **An toàn kiểu dữ liệu** (Compile-time Type Safety).

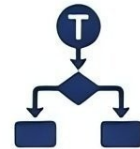


- Loại bỏ hoàn toàn ép kiểu thủ công (No Boxing/Unboxing), gia tăng hiệu năng.

Các Tập hợp Phổ biến



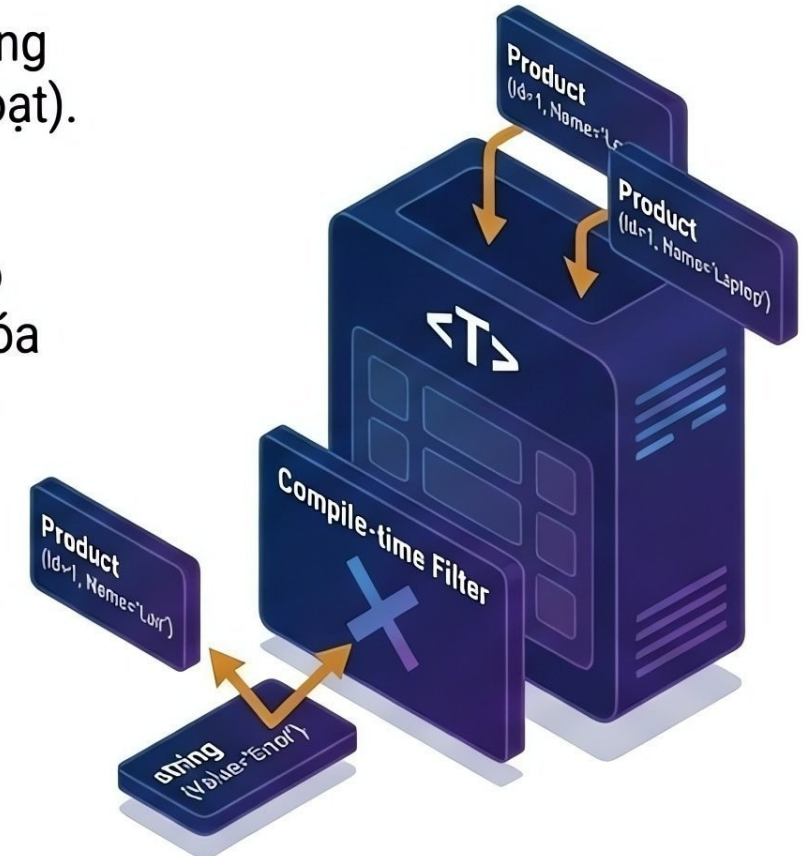
List<T>: Danh sách động (Thêm, sửa, xóa linh hoạt).



IEnumerable<T>: Giao diện chỉ đọc (Tối ưu hóa bộ nhớ khi trả dữ liệu).



Dictionary<TKey, TValue>: Cấu trúc tra cứu siêu tốc theo khóa định danh.



LINQ Cơ bản: Truy vấn & Lọc Dữ liệu trong Bộ nhớ

Concept Definition

Language Integrated Query (LINQ): Cú pháp khai báo trực quan để lọc dữ liệu chỉ bằng một dòng lệnh.



Nullable Reference Types & Xử lý Ngoại lệ An toàn



Clean Code Conventions theo Chuẩn Microsoft

Quy tắc Đặt tên Nhất quán (Naming Conventions)

✓	Lớp & Phương thức: PascalCase (VD: ShoppingCartState, CalculateTotal)
✓	Tham số & Biến cục bộ: camelCase (VD: productId, newQuantity)
✓	Interface: Bắt buộc tiền tố I (VD: IProductRepository)

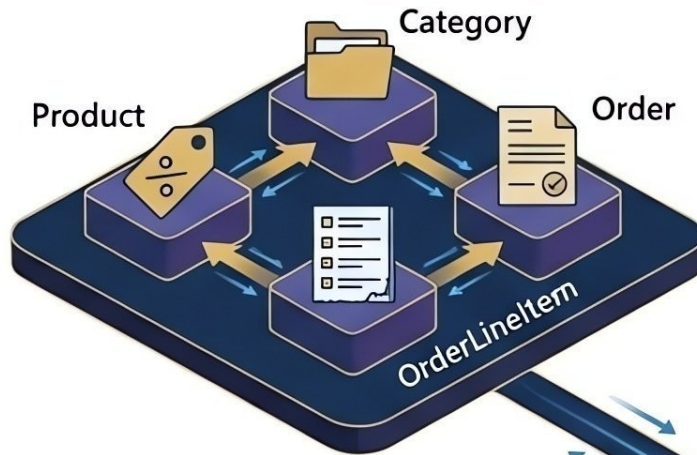
Nguyên lý Đơn nhiệm (Single Responsibility)

- ✓ Mỗi lớp chỉ đảm nhận duy nhất một khái niệm nghiệp vụ.
- ✓ Phương thức tinh gọn, tối đa 20-30 dòng lệnh xử lý tập trung.



Tổng kết Kiến trúc: Bộ Domain Entities Cốt lõi

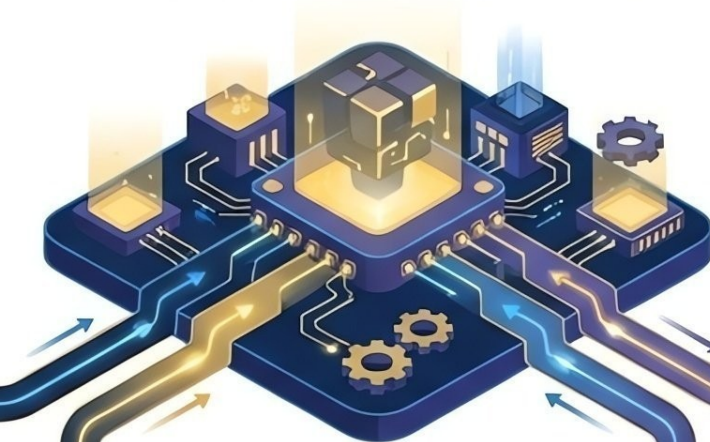
4 Thực thể Trọng tâm



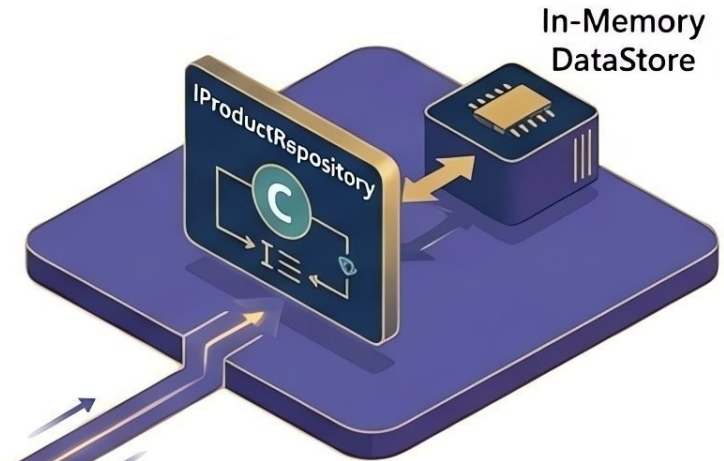
Thiết kế hoàn chỉnh Product, Category, Order, OrderLineItem.

eShop.CoreBusiness

Toàn bộ mã nguồn nền móng được đóng gói an toàn tuyệt đối trong project trung tâm.



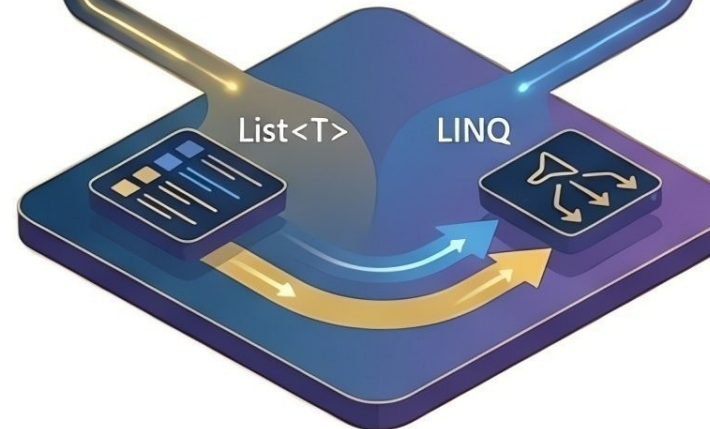
Tách rời Logic



Bọc qua IProductRepository, sẵn sàng cho In-Memory DataStore.

Thao tác Dữ liệu

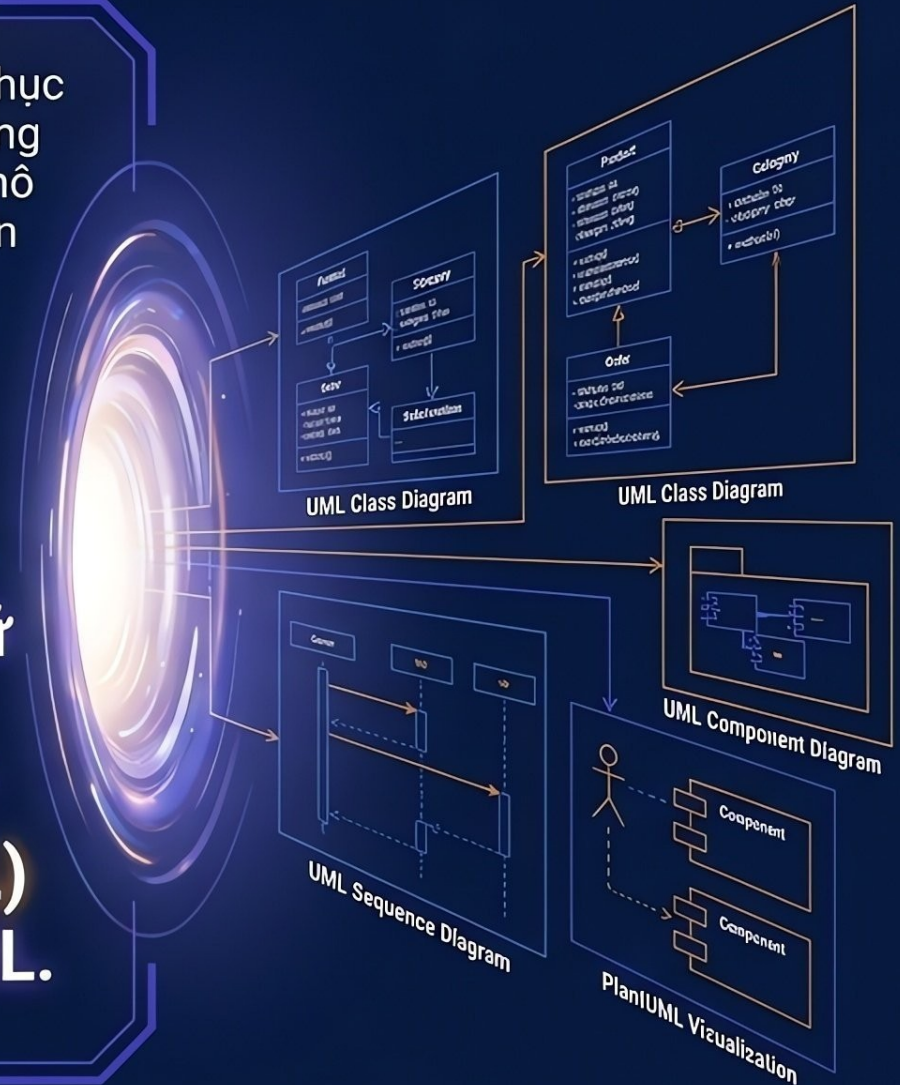
Xử lý mượt mà bằng Generics (`List<T>`) và LINQ.



Thách thức Kiến trúc Tiếp theo

Khi dự án phình to thành hàng chục lớp đối tượng và các luồng tương tác phức tạp, làm thế nào để mô hình hóa toàn bộ hệ thống trên một bản vẽ duy nhất mà mọi kỹ sư đều hiểu?

Giải pháp đón chờ:
Sử dụng Ngôn ngữ
Mô hình hóa
Thống nhất ngữ
Thống nhất (UML)
& Công cụ PlantUML.



Mô hình hóa Hệ thống Web với Ngôn ngữ UML

Mục tiêu Cốt lõi

Chuyển hóa yêu cầu nghiệp vụ thành bản thiết kế kỹ thuật chuẩn xác.

Ngôn ngữ Quốc tế

UML (Unified Modeling Language): Tiếng nói chung của kỹ sư phần mềm doanh nghiệp.



3 Sơ đồ Cốt tử

1. **Use Case:** Xác định phạm vi chức năng
2. **Class:** Cấu trúc dữ liệu tĩnh
3. **Sequence:** Luồng tương tác động

Tư duy Kỹ sư: Vẽ Thiết kế Trước — Viết Code Sau



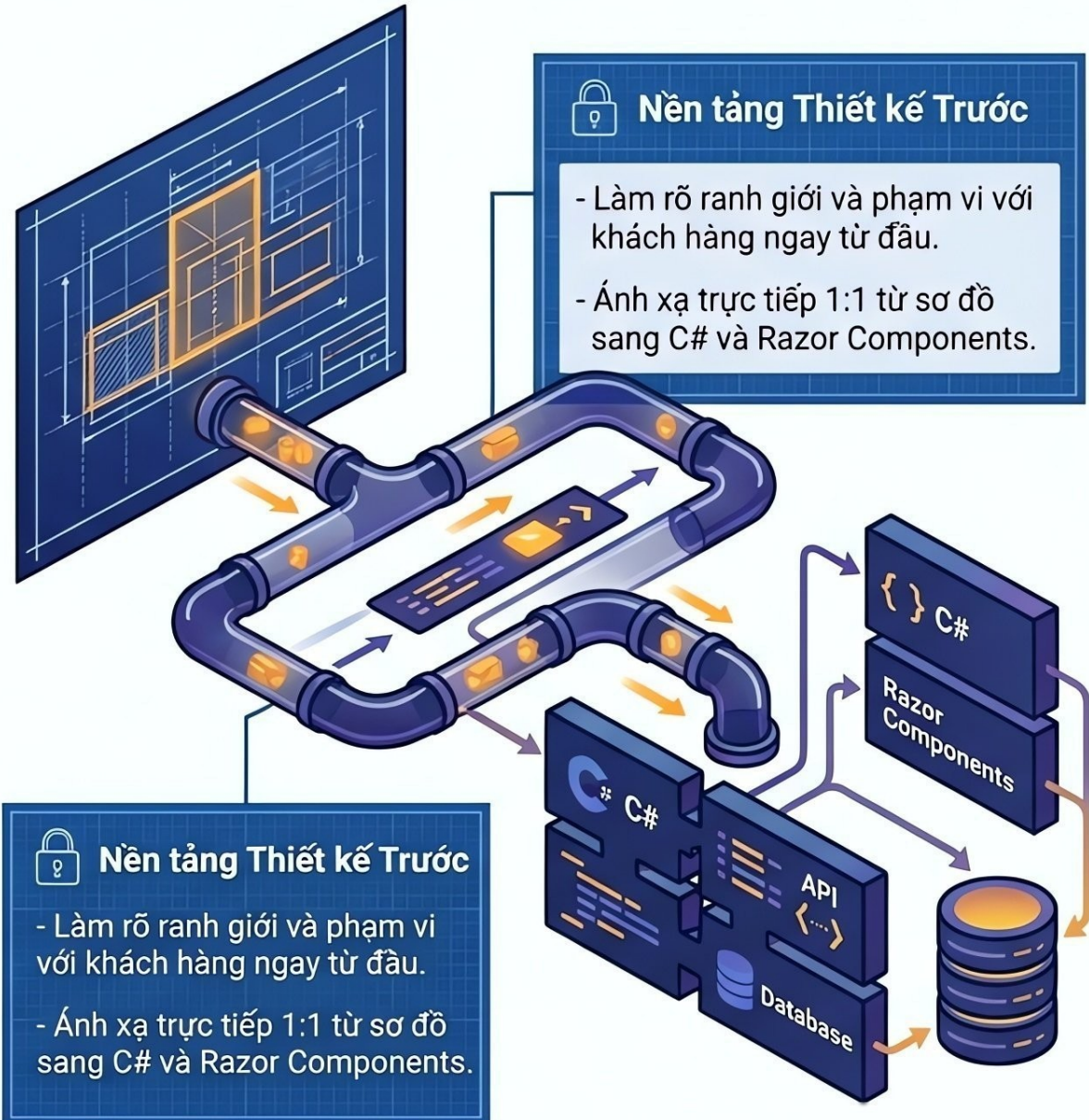
Rủi ro Code Trước - Nghĩ Sau

- Đập đi xây lại liên tục; cấu trúc cơ sở dữ liệu bị chấp vã.
- Chi phí sửa lỗi cao gấp 10 lần so với giai đoạn thiết kế.



Nền tảng Thiết kế Trước

- Làm rõ ranh giới và phạm vi với khách hàng ngay từ đầu.
- Ánh xạ trực tiếp 1:1 từ sơ đồ sang C# và Razor Components.



Nền tảng Thiết kế Trước

- Làm rõ ranh giới và phạm vi với khách hàng ngay từ đầu.
- Ánh xạ trực tiếp 1:1 từ sơ đồ sang C# và Razor Components.

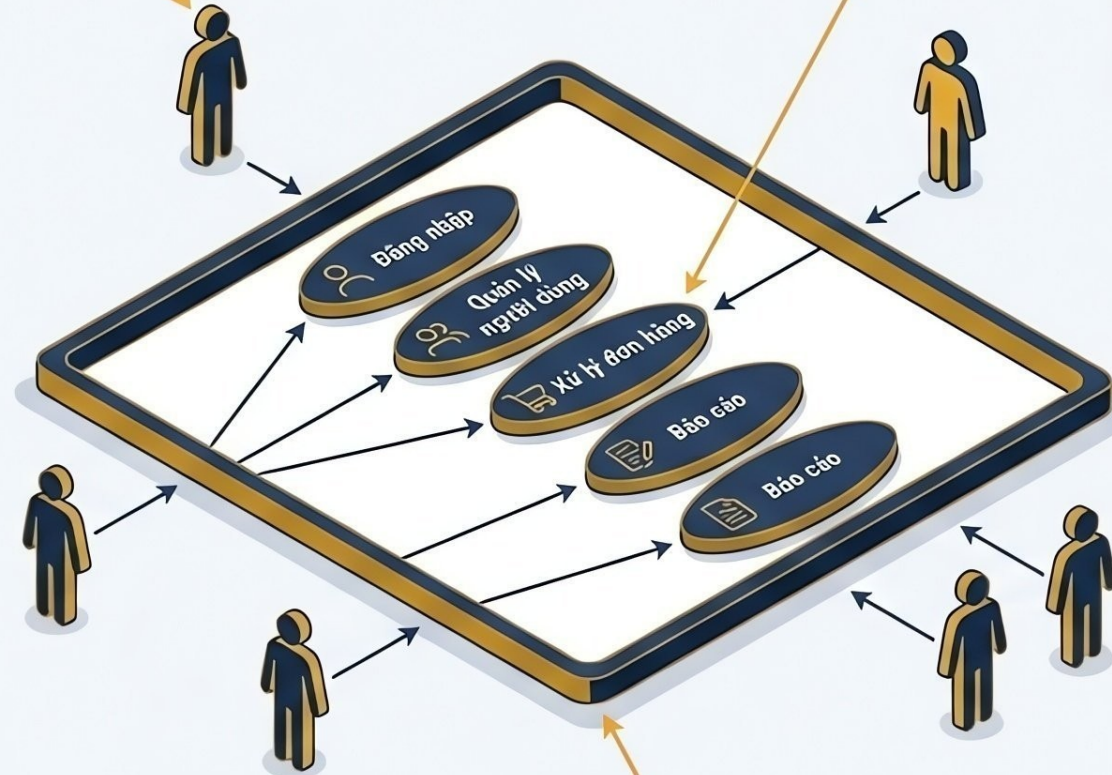
Sơ đồ Ca sử dụng (Use Case): Xác định Phạm vi

Tác nhân (Actor)

Con người hoặc hệ thống bên ngoài.
Trả lời: Hệ thống phục vụ ai?

Ca sử dụng (Use Case)

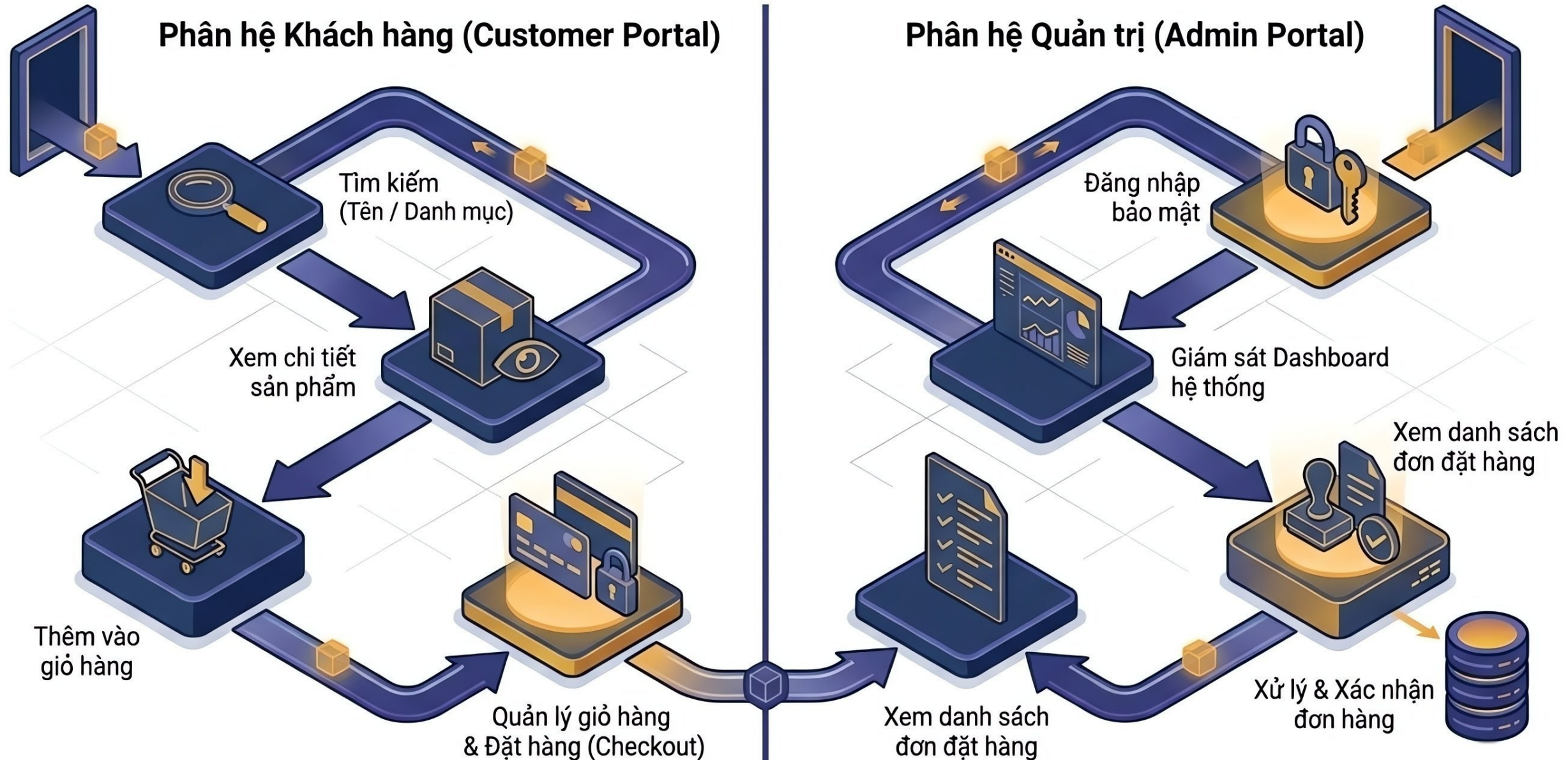
Chức năng mang lại giá trị cụ thể.
Trả lời: Hệ thống cung cấp tính năng gì?



Ranh giới (System Boundary)

Khung xác định phạm vi. Phân định rạch ròi hệ thống xử lý nghiệp vụ gì bên trong và bỏ qua cái gì bên ngoài.

Phân tích Use Case eShop: Customer vs. Admin Portal

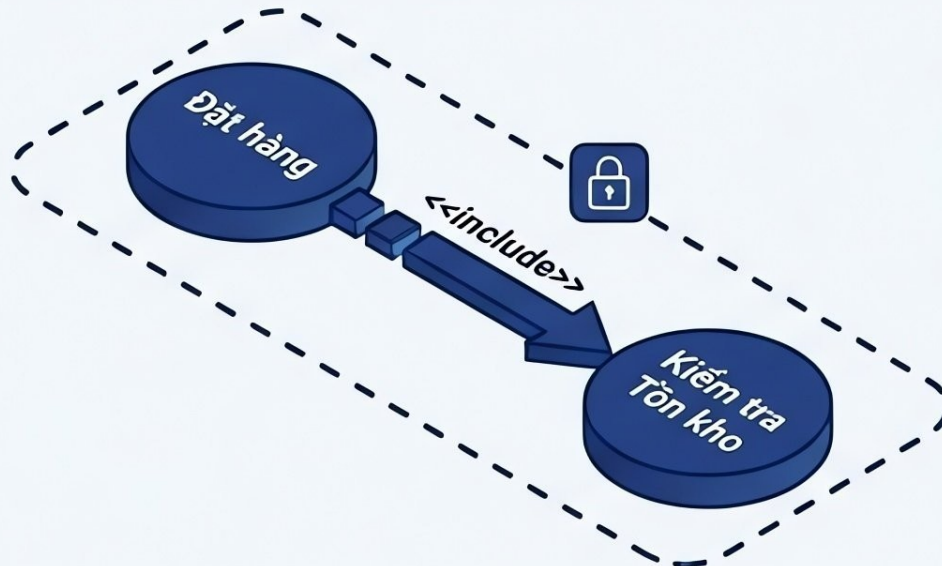


Quan hệ <<include>> và <<extend>> trong eShop

Quan hệ **Bao gồm** (<<include>>)

Tính chất: **BẮT BUỘC**

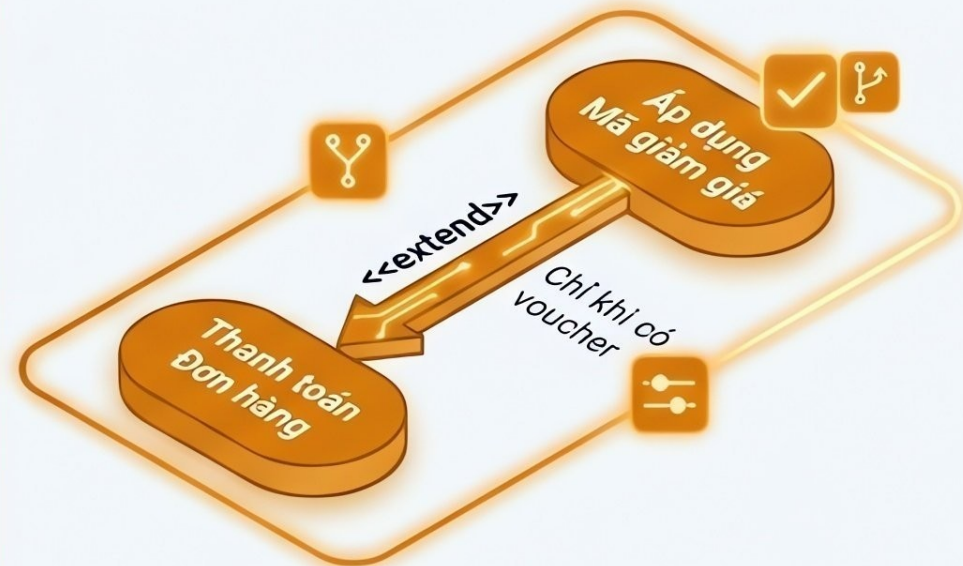
Logic: Ca sử dụng chính bắt buộc phải chạy
Ca sử dụng phụ để hoàn thành nghiệp vụ.



Quan hệ **Mở rộng** (<<extend>>)

Tính chất: **TÙY CHỌN**

Logic: Tính năng phụ rẽ nhánh, chỉ kích hoạt
khi thỏa mãn điều kiện cụ thể.



Sơ đồ Lớp (Class Diagram): Cấu trúc Tĩnh & Mối quan hệ

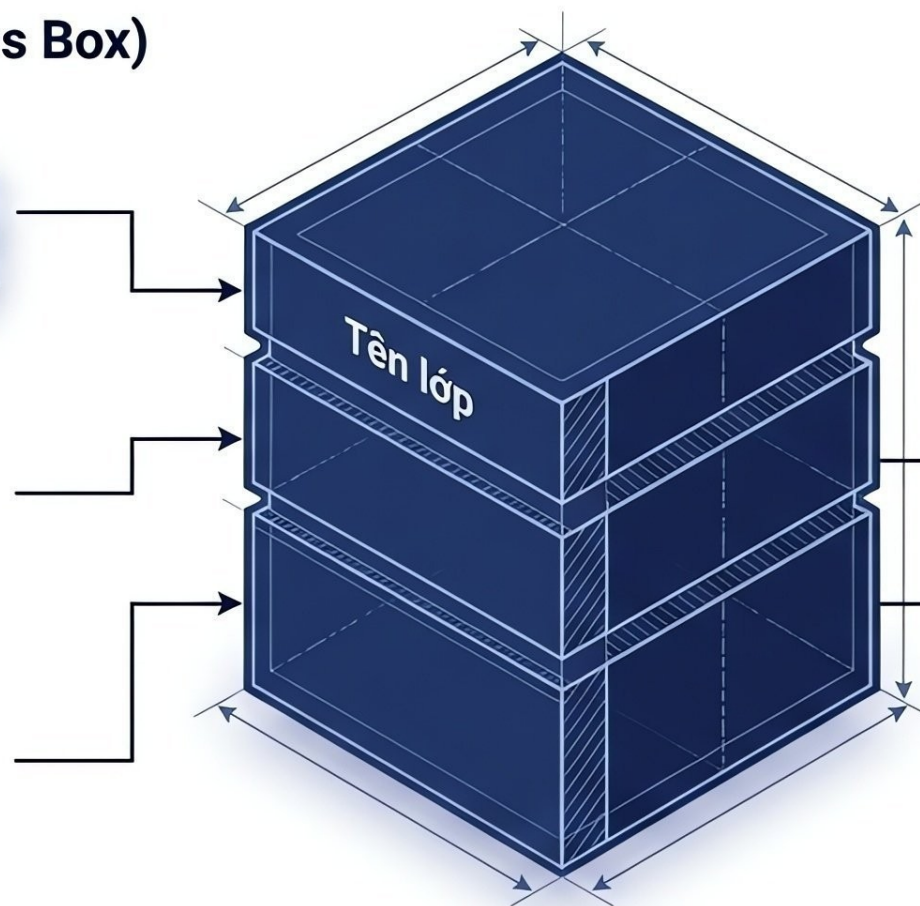
Cấu trúc 1 Hộp Lớp (Class Box)

Ngăn 1: Tên lớp
(Product, Order)

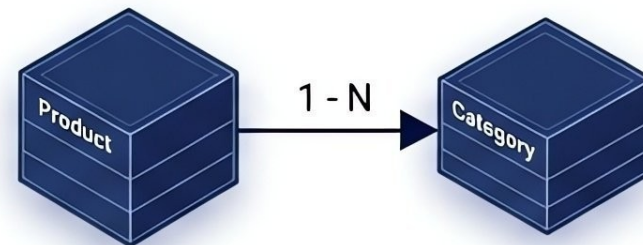


Ngăn 2:
Thuộc tính & Kiểu dữ liệu
(+ Price: double)

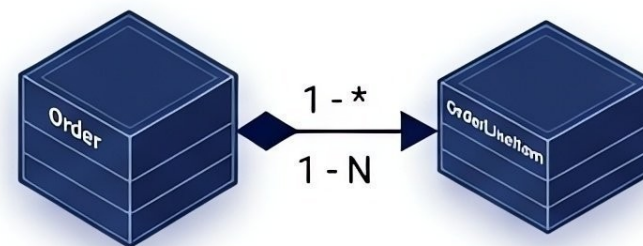
Ngăn 3:
Phương thức
(+ CalculateTotal(): double)



Các Mối quan hệ Cốt lõi

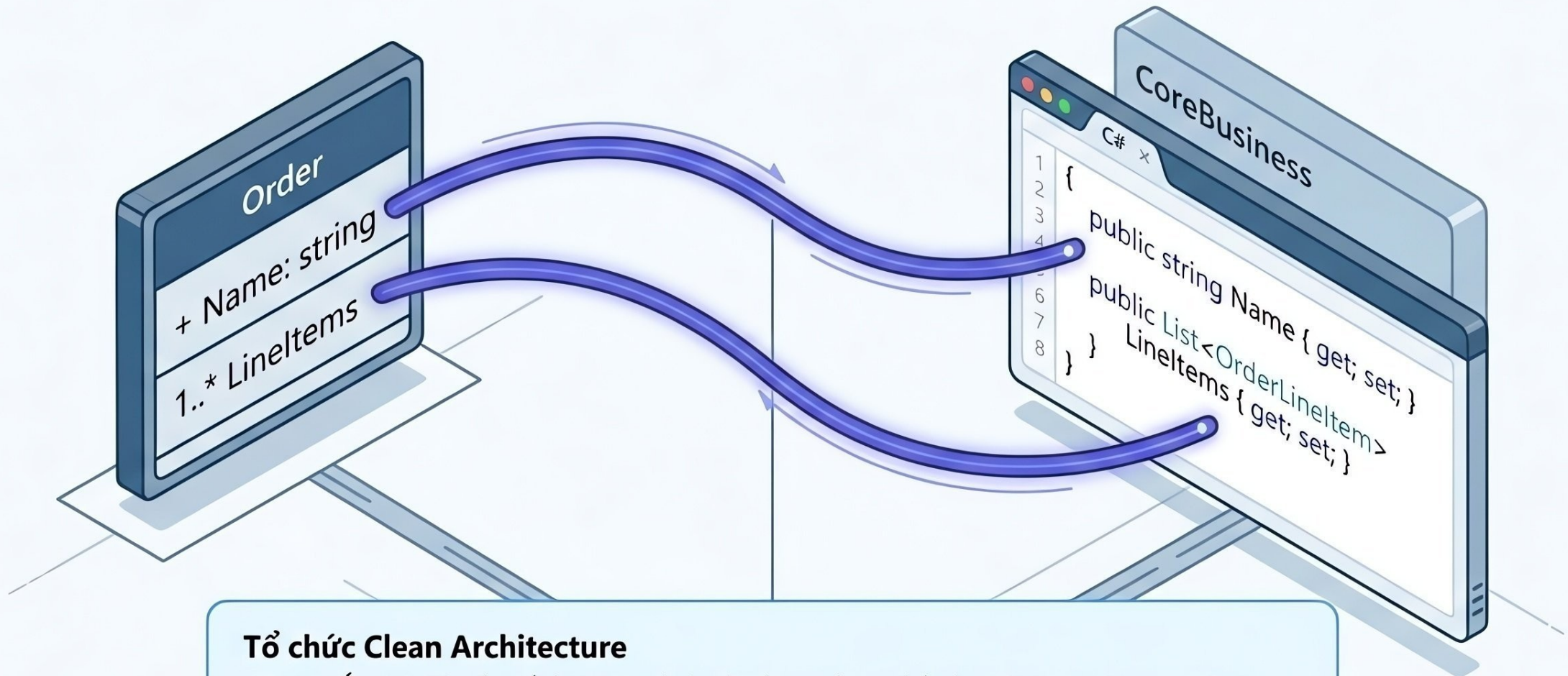


Kết hợp (Association):
Product thuộc về 1 Category
(Quan hệ 1 - N)



Hợp thành (Composition):
Order chứa nhiều OrderLineItem
(Sự phụ thuộc vòng đời: Xóa Order thì LineItem mất theo)

Ánh xạ Class Diagram sang C# Models trong CoreBusiness

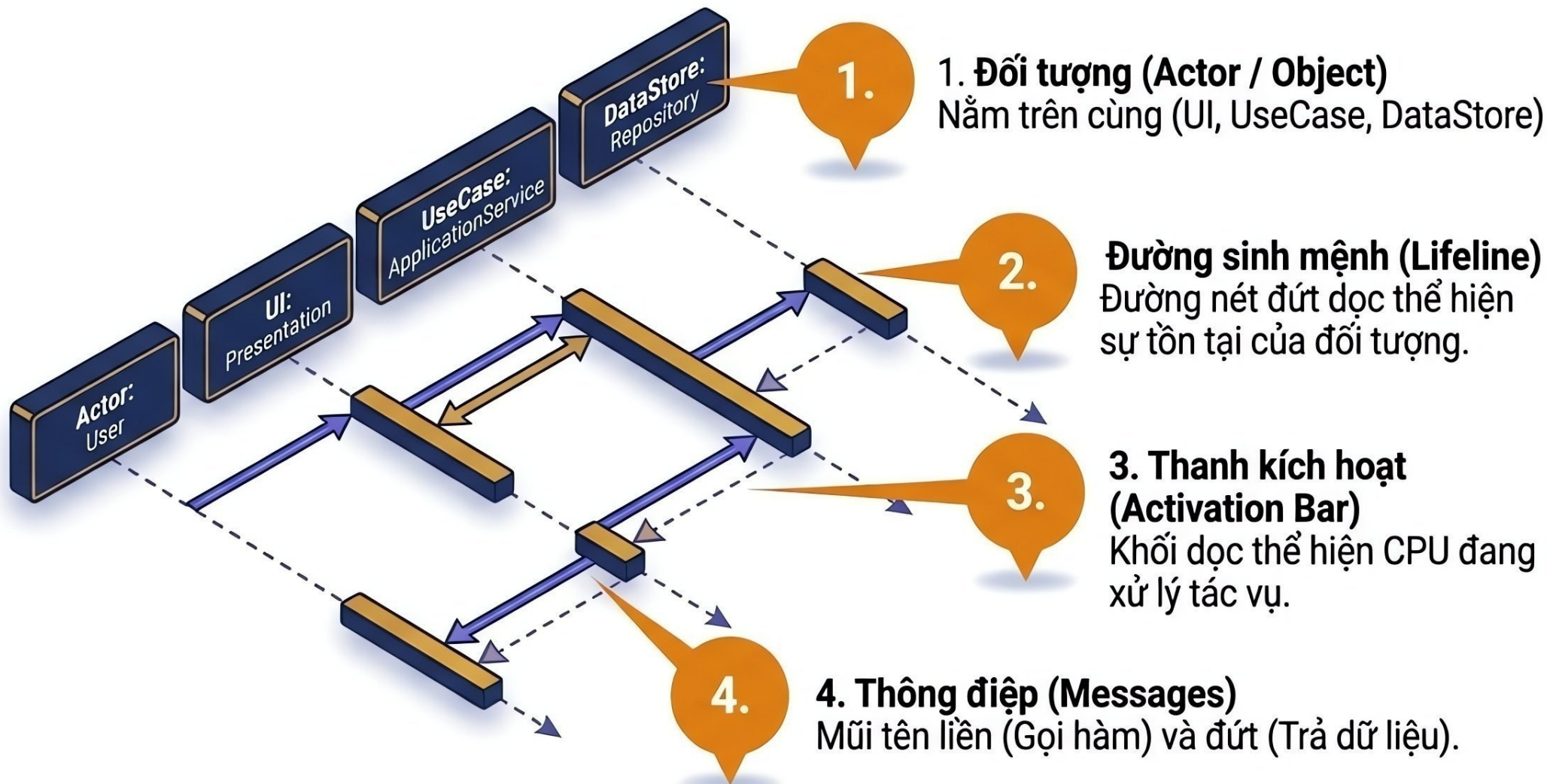


Tổ chức Clean Architecture

- Quy tắc 1:1: Thuộc tính UML dịch thuật cơ học thành **C# Auto-Properties**.
- **Zero Dependencies**: Tầng Domain (eShop.CoreBusiness) thuần khiết, tuyệt đối không tham chiếu UI hay Database.

Sơ đồ Tuần tự (Sequence Diagram): Luồng Tương tác Động

Mục đích: Trực quan hóa các thông điệp trao đổi xuyên qua các tầng Clean Architecture theo trục thời gian.



Mở xẻ Sequence Diagram: Luồng Tìm kiếm Sản phẩm



Bước 1: User Interaction

Khách hàng gõ từ khóa vào
SearchBarComponent.

Bước 2: Event Binding

UI kích hoạt
ViewProductsUseCase.Execute(filter). Tuyệt đối không gọi
thẳng Database.

Bước 3: Data Retrieval

UseCase gọi
IProductRepository.GetProducts(filter) từ bộ nhớ đệm
(In-Memory).

Bước 4: Render UI

Tập hợp dữ liệu
IEnumerable<Product> chảy
ngược về và hiển thị lên
ProductItemComponent.

Mở xẻ Sequence Diagram: Luồng Thêm vào Giỏ hàng

1. Hành động Khách hàng



Bấm nút Add to Cart trên giao diện ViewProductComponent.

2. Gọi Logic Nghiệp vụ



Thực thi hàm `AddProductToCartUseCase.Execute(productId)`.

Logic nghiệp vụ xử lý yêu cầu thêm sản phẩm vào giỏ hàng.

3. Đột biến Trạng thái (State Mutation)



Cập nhật trạng thái thông qua `IShoppingCart.AddProduct(product)`.

Trạng thái giỏ hàng được cập nhật trong bộ nhớ (In-Memory).

4. Quản lý Trạng thái Thời gian thực



Tự động bắn sự kiện cập nhật biến đếm số lượng trên biểu tượng Giỏ hàng tại MainLayout (Không tải lại trang).

Trạng thái giao diện được cập nhật ngay lập tức nhờ Blazor State Management.

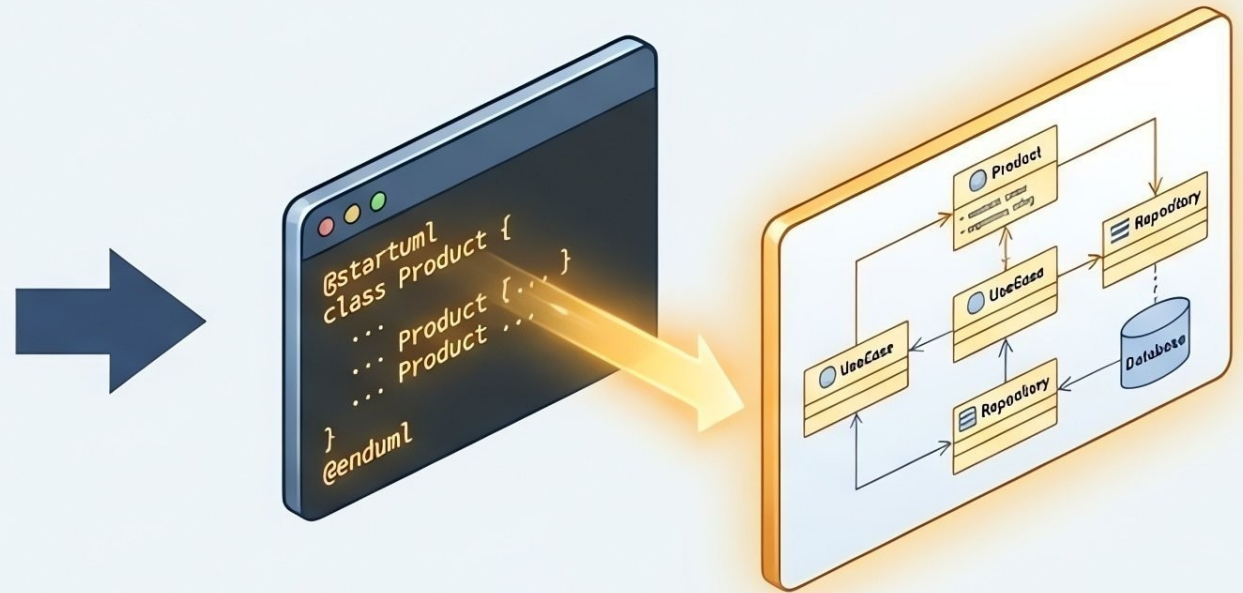
Công cụ Hiện đại: Viết UML bằng Code với PlantUML

Phương thức Cũ:
Kéo Thả (Visio, Draw.io)



- Công kênh, mất thời gian căn chỉnh thủ công từng pixel.
- Không thể lưu trữ và quản lý phiên bản lịch sử thay đổi bằng hệ thống Git.

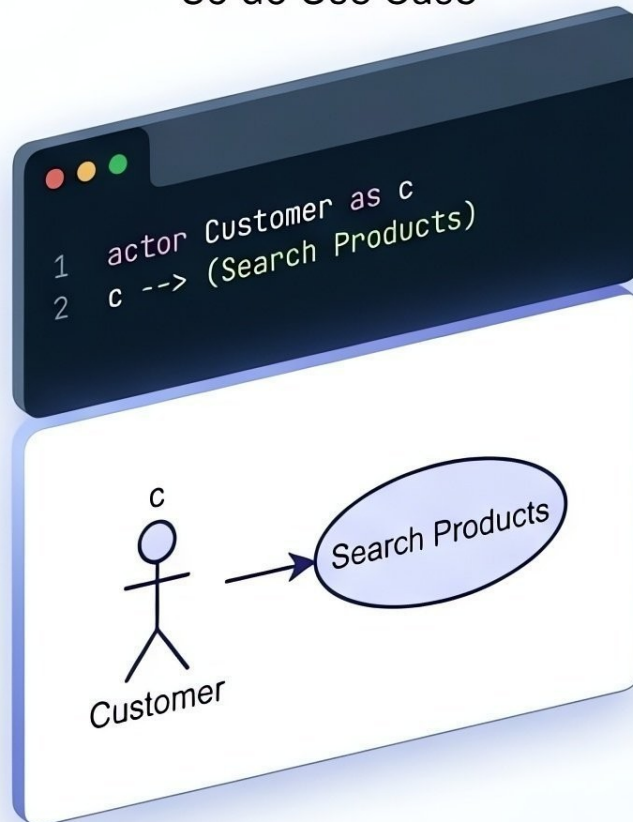
Xu hướng Mới:
Diagrams as Code (PlantUML)



- Viết bản vẽ bằng văn bản thuần túy (*Text-based syntax*), tốc độ cực nhanh.
- Tự động sinh hình vẽ kỹ thuật chuyên nghiệp, nhúng trực tiếp vào Markdown.
- Quản lý phiên bản (Git diff) dễ dàng và đồng bộ tuyệt đối với mã nguồn C#.

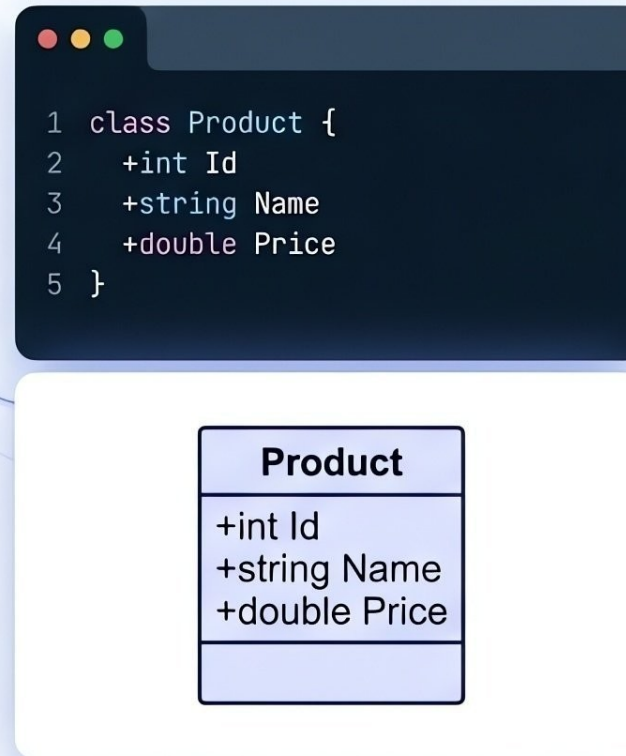
Cú pháp PlantUML Nhanh: 3 Sơ đồ trong 5 Phút

Sơ đồ Use Case



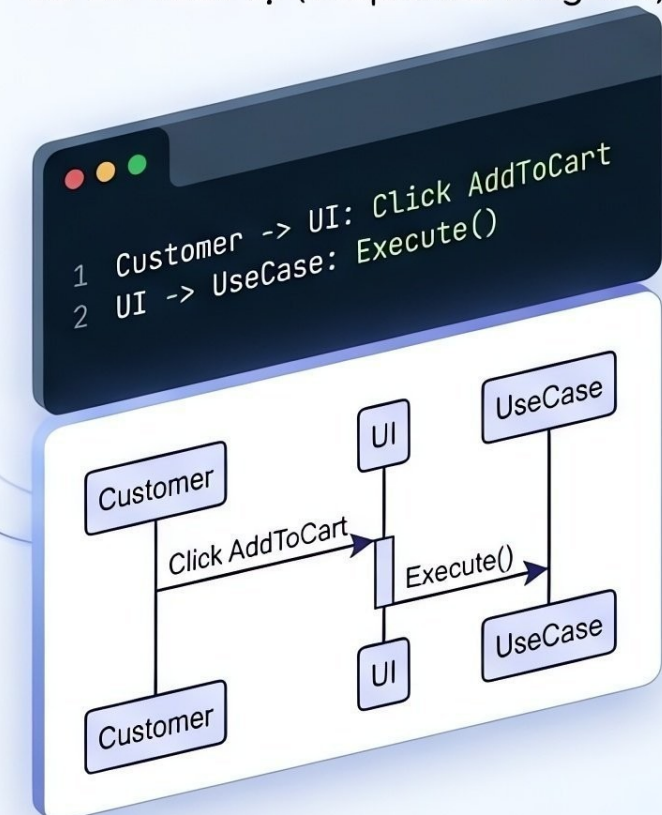
Sinh ra Tác nhân trả mũi tên vào chức năng.

Sơ đồ Lớp (Class Diagram)



Sinh ra khối Class Box 3 ngăn.

Sơ đồ Tuần tự (Sequence Diagram)

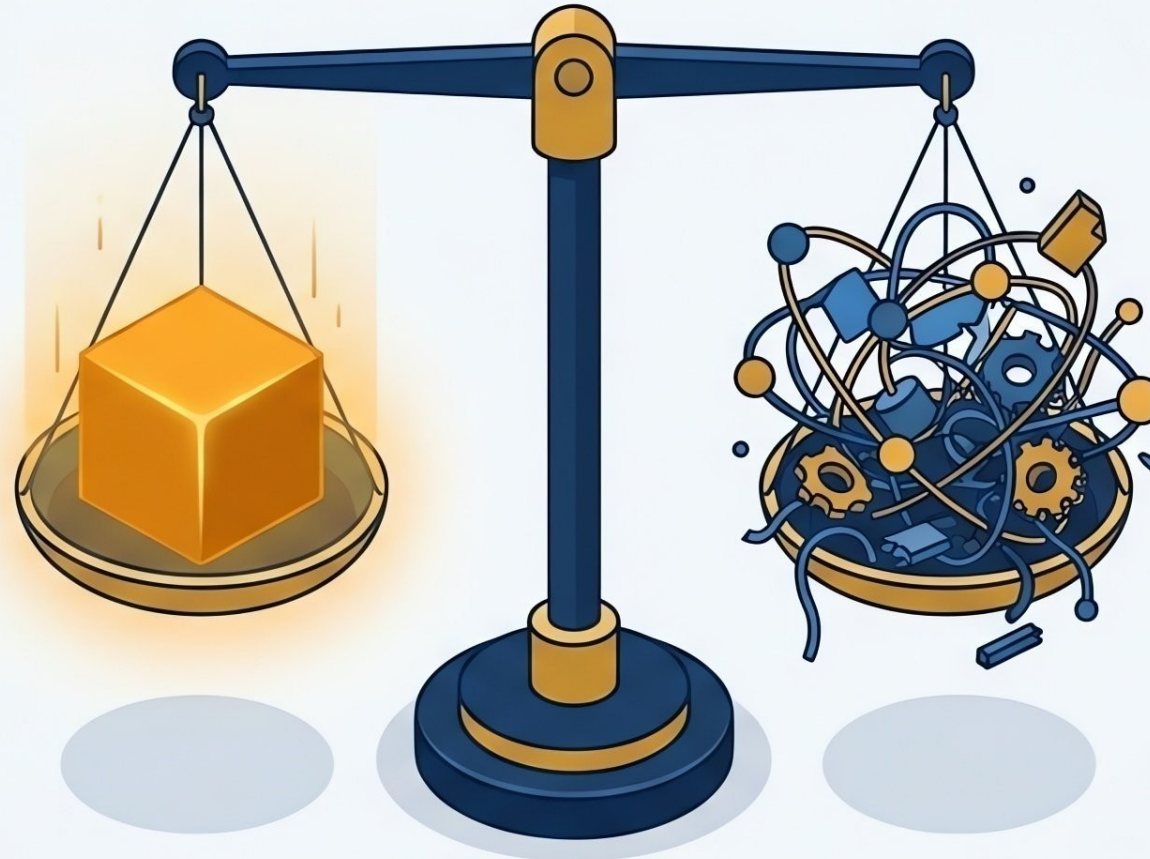


Sinh ra trục Lifeline và thanh thông điệp.

Tránh Bẫy Thiết kế Quá đà (Over-Engineering)

Nguyên lý YAGNI (You Aren't Gonna Need It)

- Chỉ tập trung xây dựng chính xác những tính năng hệ thống đang thực sự cần.
- Tuyệt đối không vẽ hàng trăm lớp phức tạp cho các kịch bản "có thể sẽ cần trong tương lai".



Nguyên lý KISS (Keep It Simple, Stupid)

- Một sơ đồ lớp xuất sắc là một sơ đồ đơn giản, trực diện và dễ bảo trì.
- Ưu tiên sự rõ ràng của luồng nghiệp vụ thay vì phô diễn công nghệ.

Bộ Khung Thiết kế Hoàn chỉnh cho Kỹ sư Doanh nghiệp

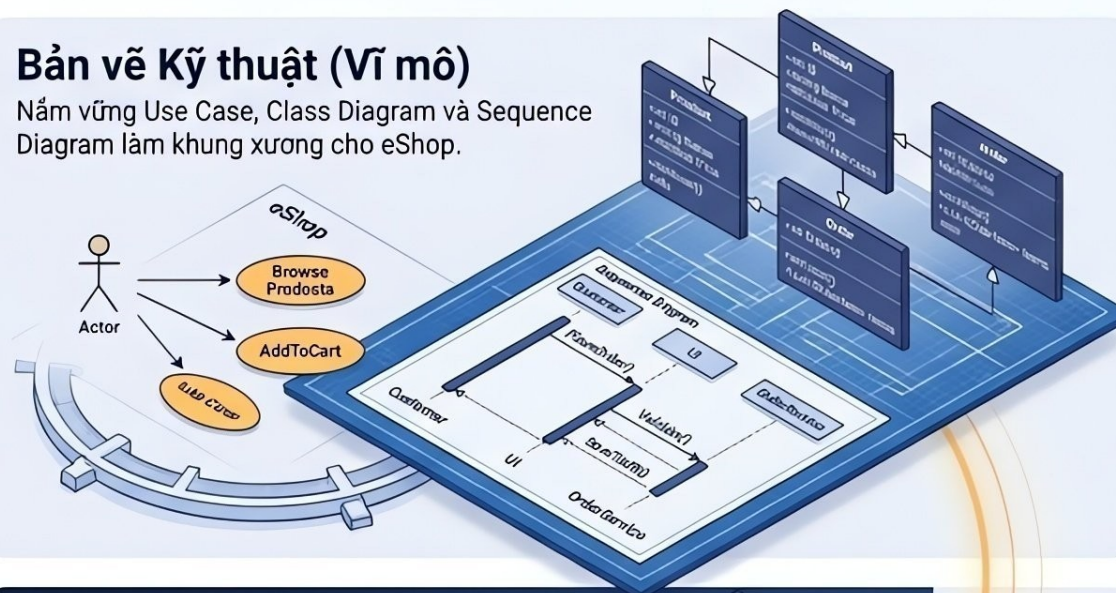
Tầng Thực thể (Vi mô)

Triniting of a matorratic center for bum Enterprise Architect.



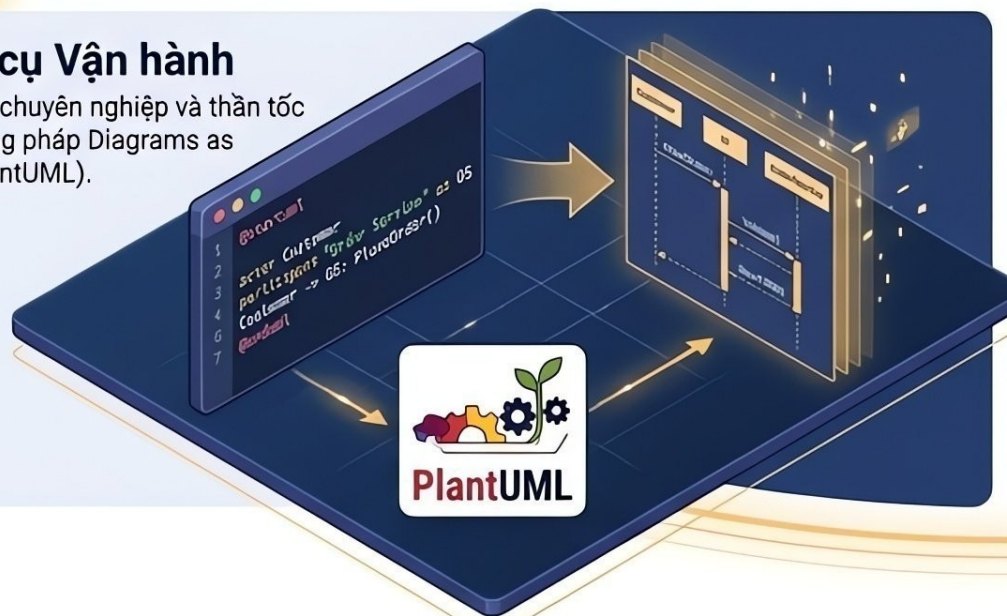
Bản vẽ Kỹ thuật (Vĩ mô)

Nắm vững Use Case, Class Diagram và Sequence Diagram làm khung xương cho eShop.



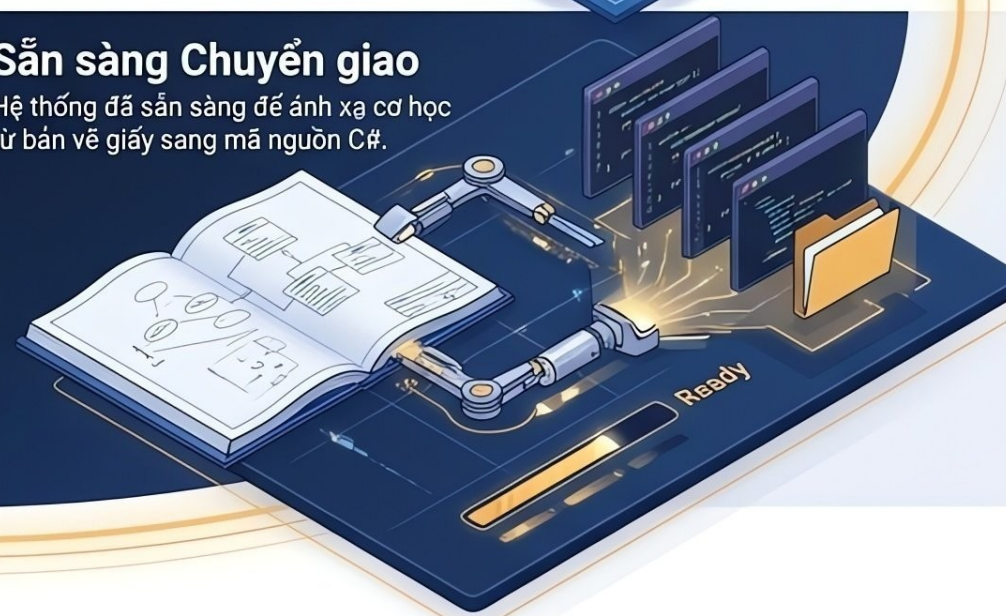
Công cụ Vận hành

Trình bày chuyên nghiệp và thần tốc với phương pháp Diagrams as Code (PlantUML).



Sẵn sàng Chuyển giao

Hệ thống đã sẵn sàng để ánh xạ cơ học từ bản vẽ giấy sang mã nguồn C#.



Nhiệm vụ Thực hành: Chuyển hóa Bản vẽ thành Mã nguồn

LAUNCHPAD CHECKLIST



1. Bắt buộc: Hoàn thành Video Part 1 (10 Videos)

- Tự tay dựng giao diện eShop, lập trình SearchProductComponent, và cài đặt In-Memory DataStore.



2. Nền tảng: Xem trước Video Part 2 (Blazor In-Depth)

- Nắm vững cơ chế truyền tham số, sự kiện (EventCallback), xử lý Form và Dependency Injection.



3. Thực hành Kiến trúc: Milestone 1

- Tự viết mã PlantUML mô hình hóa độc lập tính năng Giỏ hàng chuẩn bị cho báo cáo cáo đồ án.

